

Chapter 1

Data as the Basis of Models



Keywords Data Mining • CRISP-DM • Scales

Machine learning has achieved success in many applications in recent years. The possibilities it opens up for us are numerous. Novel photo filters calculate our face to look younger or ensure that we always look into the camera lens. Knowledge is accessed faster than before through intelligent chatbots. In this sense, data-based models and learning methods are currently changing our environment sustainably. Many people react with skepticism. To some extent, this is justified, because artificial intelligence (AI), which is often mistakenly equated with its subfield of machine learning, cannot solve every problem. The expectation of AI is exaggerated and utopian (Fig. 1.1).

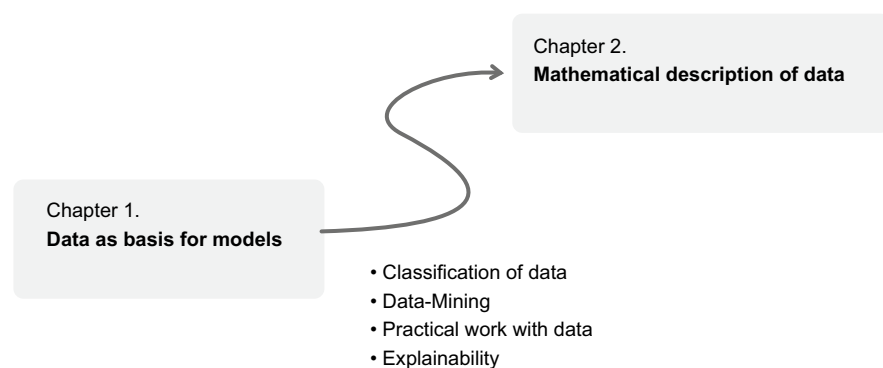


Fig. 1.1 Chapter overview

Data is the fuel for learning methods. It is important to understand how data can be characterized. This chapter therefore shows how we can describe and work with them in a structured way. We introduce Data Mining and the Cross Industry Standard Process for Data Mining (CRISP-DM) as systematic processes, with a clear focus on the practical aspects of industrial implementation. To understand the program codes in the book, we briefly introduce the programming language Python and explain its most important libraries.

Machine learning for engineering sciences focuses on the context of technical processes and production systems. It therefore primarily has a practical reference. We systematically present the topic in the context of this book and place great value on clear examples and comprehensible practical applications. Some of these applications and examples have been simplified. We present the theory to motivate confidence in a method or to highlight a specific mode of action. However, the mathematical foundations, as shown here, do not claim to be formulated in a closed manner. Rather, they should help to make the underlying mechanisms comprehensible on a mathematical basis.

1.1 Data-Based Modeling

1.1.1 *The Concept of Model*

Why is machine learning interesting at all? For this, we need to go a bit more into details of models. In the natural and engineering sciences, there has been a deep appreciation for models for many years. They imitate reality at a lower level of detail and help us understand complex relationships. Processes become predictable through models.

First-Principle Models

Many models originate from axioms and the laws of nature. They link physical quantities with each other and thus allow a direct understanding of the relationships. Because of this property, they are called First-Principle models. They are captured by compact mathematical equations. Differential equations, conservation laws up to state models of control technology are examples of this.

A well-known first-principle model illustrates this a bit closer: the law of gravity. The force that two masses m_1 and m_2 exert on each other is proportional to the product of these masses and inversely proportional to the square of their distance r ,

$$F \propto \frac{m_1 m_2}{r^2}. \quad (1.1)$$

This model allows us to directly understand the relationship between the masses and their distance. It is also compactly formulated. Eq. (1.1) states what will happen if we, for example, double the mass m_2 . On the other hand, if we find that the force has become four times smaller than before, we can say why by measuring r and knowing m_1 and m_2 : because the radius has doubled.

Let's highlight the important features of models again: a) They help us understand complex relationships as they link variables together and b) they predict the behavior of systems.

Data-based Models

Since machines are based on scientific principles, models help us identify problems in industrial production. Many technical processes are combinations of several processes. Often, the individual processes are so complex that a complete representation is only possible to a limited extent, even with reduced first-principle models. Especially the original bottom-up approach, deriving relationships from axioms or basic laws, is difficult.

This is where data-based modeling comes into play. Data exists from many technical processes. What happens in the processes can be recorded at least within the accuracy of the sensors. Assuming all relevant data is captured and we have both the influencing variables and the size we want to understand or predict, then the actual dependency is contained in the data and can be extracted from it. This is the basic assumption of data-based modeling.

In contrast to deriving a model with the bottom-up approach, data-based modeling begins with the observation of the processes. This is top-down, as the process serves as the starting point and the details are only discovered afterwards. Subsequently, statistical tools are used to set up simple models. As complexity increases, machine learning methods are used. They form a separate subgroup of data-based modeling. Yet, there are difficulties and open questions:

- **Choice of variables.** Is the dynamics we want to capture even captured by the data?
- **Quality of measurement.** Are the sensor data accurate enough to represent the problem?
- **Amount of data.** Do we have enough measurement points available to set up the desired model?

1.1.2 White-Box, Grey-Box, and Black-Box Models

The Characterization in First-Principle and data-based models is oriented towards the model's origin. Another property is the traceability of a model. Here, the following categories are distinguished:

- **White-Box Models.** Any model that is completely traceable and explainable is called a White-Box model. We can look into the model and understand how it works. First-Principle models are White-Box models. Data-based approaches such as linear regression can also be White-Box models.
- **Black-Box Models.** If traceability is not possible and thus the internal mechanisms are not known, then it is called a Black-Box model. Such models can predict processes, but they do not allow any statement about the relationship

between input and output variables. Consequently, it is difficult to trust Black-Box models. Machine learning algorithms are often accused of belonging to this category. However, this is not necessarily true. There are methods, as we will learn later, that help us to investigate models for their internal mechanisms and thus move from the Black-Box character to a real understanding and trust.

- **Grey-Box Models.** A third variant is Grey-Box models, which are partially traceable. They use input variables whose influence is known, and additional variables whose effect cannot be captured. Monte Carlo methods are counted in this category because they contain an analytical core, e.g., a differential equation, and simulate this with stochastic variables. Since the latter are random processes, at least part of the Monte Carlo simulation is unpredictable.

1.1.3 Criticism of Data-Based Modeling

The lack of reference to natural laws is a disadvantage of data-based modeling. Some practitioners and users criticize data-based models because of their difficult traceability. They base their criticism on a purely Black-Box character. The acceptance of machine learning was initially affected by this. However, many of these critics are based on false assumptions.

The diversity of models has increased over the years. Machine learning algorithms can be made understandable and explainable. They now belong to the area of Grey-Box models. The integration of physical First-Principle models as part of data-based models is unknown to many users. However, these approaches have many successes in the industry to show.

Learning methods can be sampled. By deliberately disturbing the input variables, one can check how the algorithm behaves. Thus, we can identify analytical relationships for non-deterministic neural networks. For most learning methods, a basis for explainability can be achieved with the help of tools from stochastic.

1.2 Classification of Data

1.2.1 Historical Classification of the Term Data

Handwritten lists, analog photos, post-its, or books are referred to as data—a term that is neither new nor a modern invention. It goes back several hundred years and is closely linked to the concept of number and the concept of measuring and comparing.

People began early to introduce comparative measures to keep track of agricultural activities. They could determine the stock of a cornfield and estimate the possible harvest yields, which was important for food production. By comparing

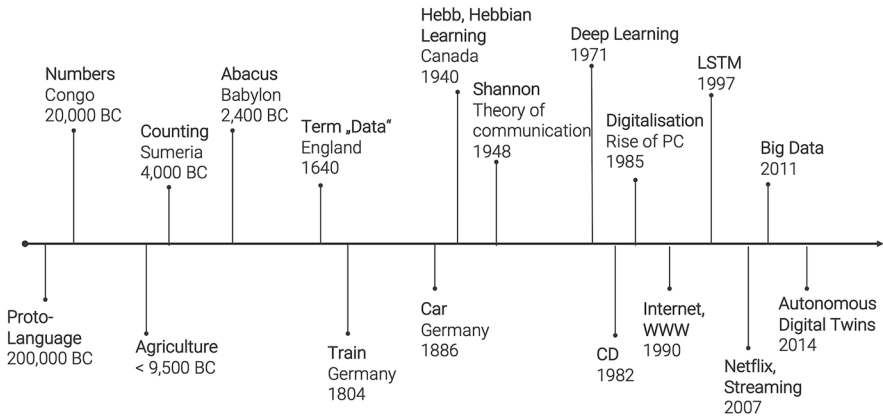


Fig. 1.2 Timeline with some important events

with the written information from previous years, it could be determined whether the yield was declining or not.

More generally, data is collected with the aim of monitoring states or processes and ultimately influencing them. In Fig. 1.2, some important events are shown to give an impression of how data has also influenced important milestones in industrial development. Please note, however, that this timeline is not complete.

1.2.2 Data in Our Present Time

Data is everywhere today. It is generated intentionally or unknowingly, planned or accidentally. Often, a lot of data is created simply because it was not considered in advance which data is really needed to be collected—an approach that is not recommended, but often unavoidable. We often simply do not know which information will be important later on. That is the reason why many data is actually never used.

We want to systematically approach the topic of data evaluation. For this, we need a basic understanding of how to describe, characterize, and structure data.

In today's time, data is ubiquitous because it is the backbone of many everyday technologies. How different data can be and that there are various types of them seems evident just by looking at text messages, websites, smartphones, or medical records. Consider what data is relevant in the following processes:

- Smartphones, which guarantee ubiquitous access to the World Wide Web,
- Social media like Facebook, which represent data collections of their users,
- Services for sending messages.

For these examples, heterogeneous combinations of different data types are always needed. In the next sections, we therefore explain how to describe and arrange data.

1.2.3 *Structural Classification*

A first, rough division of data can be made based on their structuring.

Structured Data

Each of us has used a table before. They have column names and a defined row structure. Their size is usually fixed and the type of each field is precisely defined. This type of data is referred to as structured.

Example: Data from a group of people

Imagine you have a group of 30 people. You ask them to identify themselves. To do this, you ask each person to enter certain information about themselves into a questionnaire with fixed input fields. The fields ask for things like name, date of birth, and address. Anyone who fills out the form is automatically forced to adhere to this structure. ◀

Structured data is therefore characterized by the presence of such a predefined schema, which is also referred to as a **data model**.

Further examples of structured data are:

- Data from technical processes,
- Bank transactions,
- Data for search engine optimization.

Unstructured Data

If there is no data mode, then we speak of unstructured data. These are defined as information that cannot be mapped by a fixed schema. So, you don't know for sure in what form the data will be presented. Staying with the above example, this corresponds to the case where you ask all 30 people in the room to give you some personal information. Some will start writing something, others will give you a copy of their ID card, and others will simply contribute a few personal photos. The data is unstructured.

- Photographs,
 - Books,
 - Health records,
 - Poetry albums
- are examples of unstructured data.

Semi-structured Data

Now there is another, often encountered type of data, which can be seen as a mixture of both of the above extreme cases: semi-structured data. Here,

some predefined basic form exists, so to speak a metamodel, according to which one expects the data to be structured, but in some subdivisions, one deliberately allows completely unstructured data (Fig. 1.3).

In our above example, the group of people would be asked to fill out a form with predefined fields and fields to be edited freely. So they would give their names and addresses, but also have some space to enter any additional information. Some might list their hobbies, others add three pictures, and others draw something. Part of the data is thus structured, another part unstructured. This type of data is then referred to as semi-structured.

Some examples of this are:

- Emails,
- Websites,
- Tweets,
- the memory of Digital Twins.

The structure of our data determines the choice of a suitable database architecture. Thus, SQL databases can process structured data particularly well; however, they have difficulties with unstructured or semi-structured data. In contrast, there are various Not-Only-SQL-(NoSQL)-database concepts, that were specifically developed for the storage and management of semi-structured data.

1.2.4 Quantitative Categorization of Data

Data can further be divided into two more categories: continuous and discrete data.

Continuous Data

Data that can assume any value within a certain range are referred to as continuous data. Measuring a length with a ruler is an example of (quasi-)continuity. Any arbitrary number (0.1, 0.01, 0.001, ...) is theoretically possible. Sometimes continuous data is synonymously seen with the term **analog data**.

Discrete Data

Discrete data refers to individual, clearly separable data points that can be counted and are often represented by whole numbers (0, 1, 2, 3, 4, 5,...). An example of discrete data is the number of people on a train. Since there is no such thing as half a person, this can only be described with discrete numbers. **Digital data is discrete.**

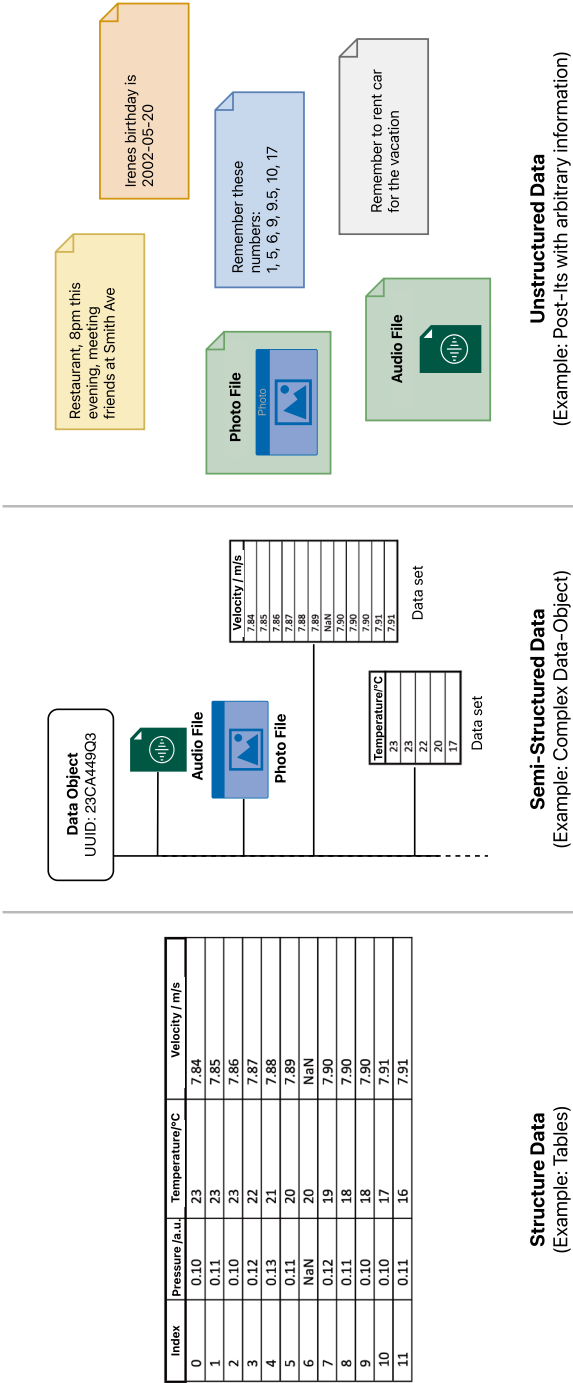


Fig. 1.3 From left to right: Structured, semi-structured, and unstructured data

1.2.5 Qualitative Categorization of Data

We can add another level of description to better understand data. This considers the nature of the data itself: Are they letters or numbers, months or lengths? The following qualitative types can be distinguished:

Nominal or attributive data

Descriptive data is called nominal—they carry a name. Terms in lists, such as colors, street names, and descriptions, represent cases of nominal data. In programming languages, the most common type used for nominal data is the **string**. They are also difficult to measure. They often require manual input. If we consider the example with the 30 people and the forms, the entry fields first name, last name, and address are of course of a nominal nature.

Ordinal Data

If we can put the data in a logical order, they are ordered and we call this type ordinal data. Obvious ordinal data are e.g. the names of the months (“January”, “February”, ...). Please note that ordinal data also have a nominal component, as the example with the month names shows. The ability to sort is thus an additional capability of this form of data.

Cardinal Data

We can add and subtract cardinal data, multiply and divide. It is possible to apply arithmetic rules to them. Number spaces are cardinal data. With month names (which are ordinal and not cardinal), you cannot perform addition, for example. That’s why we use the separate term cardinal, (lat. *numeri cardinale*, special numbers). In programming languages, the data type **integer** represents the natural numbers, and the data type **float** (floating point) captures the real numbers (floating point numbers).

Binary Data

Binary decisions are “Yes”/“No”, 0/1 or “True”/“False”. The specific variant with “True”/“False” is also referred to as a **boolean** decision. The data type in programming languages is called **bool** or **boolean**.

Labels and Target Sizes

A very important, more general type of qualitative data are the so-called **labels**. These are a specific set of nominal, ordinal, cardinal, or binary information used in supervised machine learning as training targets for the algorithm. The term label will accompany us in further chapters; therefore, it is highlighted here. ◀

1.2.6 Time Series

A data series, in which the individual data points are indexed and ordered by time, is referred to as a time series. Time series can be categorized as cardinal, discrete data. This type of data is primarily found in technical processes. In physics, most dynamic equations (the model) are formulated as derivatives with respect to time. All these systems can be monitored by measuring time series, e.g., to verify the validity of the model. If you consider a piece of steel that cools down after heating and rolling, and you take measurements every 5 min, you get a time series.

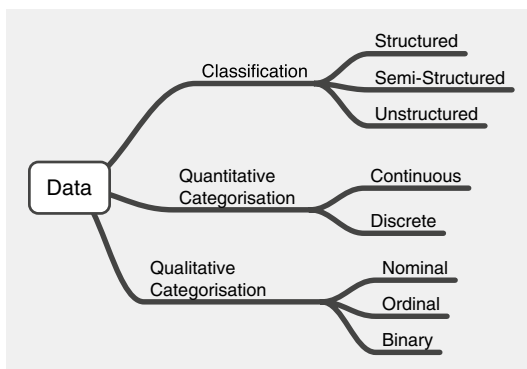
In the analysis of technical processes, the majority of cases involve combinations of time series and individual nominal attributes. They are at the heart of technical data mining (Fig. 1.4).

Time series can also be evaluated in the frequency domain, which is well suited for repetitive and oscillating processes. The scope of time series analysis becomes clear when looking at the book by J. D. Hamilton [5], in which many different model approaches to time series are discussed.

1.2.7 Scales

The consideration of the above qualitative and quantitative categorization of data is related to the concept of scale [4]. A scale divides a set of values or properties into measurable sections. It is a benchmark for our data. Depending on their qualitative characterization, we distinguish, for example, nominal scales or ordinal scales. The scale depends on its data type and the ability to divide the data collection into measurable sections. The difference between nominal and ordinal scales becomes clearer with the following examples:

Fig. 1.4 A simple scheme to describe data



Example: Teacher and Students
Imagine you are a teacher and teach a class with N students. Each student has a first name. When you write down the first names, you record nominal data. What can you measure from these data? In fact, you can determine the frequency with which a name occurs. Ultimately, you know that there are four people named Michael and two named Julia, while the remaining names occur exactly once. ◀

Example: Students' Birthdays as Temporal Ordering Variable
If you go one step further and also ask for the students' birthdays, you extend your nominal data collection by an ordinal variable, namely a date. Now you can sort the individual people by their birthday. You can even distinguish people with the same name. ◀

Nominal data can therefore only be compared with each other or with a reference. For variables of this category, you only have the operations $=$ or $\neq$ available. The ordinal data, on the other hand, also have the operations $<$ and $>$.

Consequently, for cardinal data, there is another scale, the cardinal scale. On it, mathematical operations $+$, $-$, $*$ and $/$ are executable. Since this allows the comparison of different intervals, this scale is also called interval scale. If a defined zero point is additionally introduced, we can not only define intervals, but also calculate ratios. A cardinal scale with zero point is therefore called ratio scale. Tab. 1.1 summarizes the different scales and their properties once again compactly.

Example: Analog and digital temperature measurement
When a weather station measures temperatures, it retrieves the temperature value from the sensor at fixed intervals. These are discrete, cardinal

Tab. 1.1 Overview of scale levels

Scale level	Math. Operation	Measurement sizes	Stat. Sizes	Example
Nominal scale	$=, \neq$	Frequency, Sorting	Mode	Names
Ordinal scale	$=, \neq, <, >$	Frequency	Median	Months
		Sorting		
Cardinal scale –	$=, \neq, <, >$	Frequency,	Arith.	Temperature/°C
Interval scale	$+, -, *, /$	Sorting, Distance	Average	
Cardinal scale –	$=, \neq, <, >$	Frequency, Sorting	Geo.	Temperature/ K
Ratio scale	$+, -, *, /, 0$	Distance, Zero point	Average	

data. With a liquid thermometer, this recording is on the cardinal scale, but analog. ◀

Example: Seismograph

The recording of vibrations of the earth's surface is done with a seismograph. It consists of a spring with an inert mass. As soon as the ground is shaken, a recorder records the relative movement of the ground to the mass. This is an example of continuous measurement. The data from the seismograph are continuous time series on a cardinal scale. ◀

An important task of machine learning in practice is the interplay of data on their various scales. We use our definitions from the previous sections to consider some more examples and classify them:

Example: Lottery

A weekly drawing of lottery numbers is a sample from an N -element set. The data generated in this way are discrete. It is an upwardly bounded nominal scale of defined numbers. Summation and subtraction are not meaningfully applicable here. Also, the order of the numbers to be drawn does not matter—but the order in which they are drawn does. ◀

Example: Books as a collection of data

The text in books is a discrete sequence of nominal information—captured by separate letters that form words and separate words that form sentences. ◀

Note the scales you are working on. Scales also have an influence on your interpretation of data: A change of 5 °C from 20 °C to 25 °C, corresponds in the Celsius scale (an interval scale) to a change of 25 %. In the Kelvin scale (a ratio scale with an absolute zero point) the same change from 293 K to 298 K is only a difference of 1.7 %. It may be that your own perception, however, corresponds more to the Celsius scale—because this scaling is simply better oriented to our reality of life.

This last point will be discussed again in the next chapter when we consider the normalization of data. Then we artificially intervene in the scaling of our data to optimize the scale for a problem.

1.3 Data Mining as a Systematic Process

1.3.1 What is Data Mining?

The term data mining is based on the idea of mining rock. If you think of data as a difficult, opaque material, this analogy makes sense. The corresponding methods for extracting information from this data rock are then imagined as mining processes.

The analogy to mining is indeed appealing, but it harbors an interesting discrepancy to the practice of data analysis: It assumes that the data already exists and only needs to be evaluated. In fact, however, the task of a data miner also includes defining suitable ways to obtain the data. Data mining is understood as the analysis of a data set with the following objectives:

- **Detection.** Detection means perceiving something specific, respectively noticing it. In the context of data, it is often necessary to find outliers, anomalies or other relevant and unexpected behaviors that are responsible for problems in the technical process. It is often a challenge to clearly define what the normal (desired) behavior of a system is.

Example: Anomaly in Aircraft Control

Consider the control of an airplane. It has actuators such as thrust, elevators and rudders, tailplane and flaps. A pilot can influence the movement of the aircraft with these parameters. To measure the state of the aircraft and the result of possible control inputs, sensors are also present. They measure, among other things, the pressure and can derive variables such as speed or altitude from it. If a control intervention is made, the individual sensors can react differently.

If you push the control stick forward, the aircraft tilts and the pressure begins to rise. A situation that is normal. However, if you notice a decrease in pressure with the same control input, you have an anomaly. Possibly the associated sensor is defective. ◀

- **Classification.** An important goal of data mining is to group data elements into classes. This is especially important in industrial problems. Here it happens that products have to meet quality classes or a product has to meet the acceptance criteria of a customer.

Example: Classification of Good and Bad Products

In the production of screw heads, an employee notices that the number of faulty threads has increased over the last few days. Therefore, he divides the

products into two groups, good and bad threads. He decides this based on the condition of the burr on the thread, which is a subjective criterion. ◀

The example shows the greatest difficulty at this point: the meaningful and complete definition of when something belongs to a category.

- **Agglomeration.** Uncovering hidden structures and clusters in the data is another goal of data mining. They help to develop a deeper understanding of the underlying dynamics. This often leads to the optimization of processes.

Example: Agglomeration of Good and Bad Products

We extend the last example and assume that all bad screws were produced at similar pressure values and similar temperatures. They therefore accumulate in the data space to form a so-called cluster. If we know these clusters, we can predict for future products whether the production will be successful or not. ◀

- **Association.** If dependencies are recognized in the data, rules and predictions can be derived from them. This is then referred to as association.

Example: Association of Good and Bad Products

Again, we extend the previous example. We know the parameters pressure and temperature that lead to bad products. Now we set up rules for good products from this by defining limit values for the critical variables—we therefore associate the terms good and bad with corresponding permissible data ranges. Whenever the screws come too close to these limit values or exceed them during their production, a more detailed examination is necessary. ◀

- **Regression.** As soon as the course of a variable can be predicted from the other variables, we speak of regression. It represents nothing more than a model of the process described by the variable.

Example: Exercise Bike

In the gym, there are stationary bikes. Although the bikes themselves do not move, a speed and a distance covered are determined from the rotation of the pedals. For this purpose, a real distance covered is simulated from the sensor data via regression models. ◀

1.3.2 Steps to Perform Data Mining

This objective is achieved through a specific (linear) workflow, which represents the actual definition of the tasks associated with data mining:

- **1. Focusing.** Here, the data required for the analysis are selected and collected. They are grouped and, if necessary, enriched with knowledge.
- **2. Cleaning.** Data can be distorted, contain false values or outliers. As a critical element of preprocessing, cleaning specifically also refers to techniques for dealing with incomplete data.
- **3. Transformation.** Often, data must be transformed in one way or another, with some transformation steps being necessary:
 - Rescaling, rebinning or resampling,
 - Normalization,
 - Differentiation (1st derivative, 2nd derivative, ...),
 - Integration and moving average,
 - Fourier transform (or Mellin, Laplace, Lagrange transform),
 - Wavelet transformation,
 - Determination of probability densities and histograms.
- **4. Analysis.** The most exciting part of a data mining activity is certainly the analysis itself. Here, appropriate algorithms are applied to evaluate the data. These algorithms will be explained in later chapters and include methods of analytical interpretation, supervised and unsupervised learning.
- **5. Evaluation and Reflection.** Based on the analysis result, it is necessary to critically question the result and make an assessment of its validity compared to the actual technical process.

Steps 1–3 are commonly also referred to as **Preprocessing**.

1.3.3 Drawing Conclusions from Data

The final step, the evaluation of the results, is necessary to critically reflect on the findings. In Fig. 1.5, the consumption of ice cream and the number of serial offenses are shown, normalized to a common maximum. Note the high correlation of these data, i.e., a great similarity in the shape and dependence of both variables over the course of the year. Can we already derive meaningful results from these two curves? Let's try out some causal dependencies and formulate what we could conclude from the mere observation of the data:

1. Ice cream consumption leads to violent crimes.
2. Criminals eat a lot of ice cream.

Both statements are of course misleading. Even if the data suggest otherwise, there is no causal relationship between the two processes. The misinterpretation arises

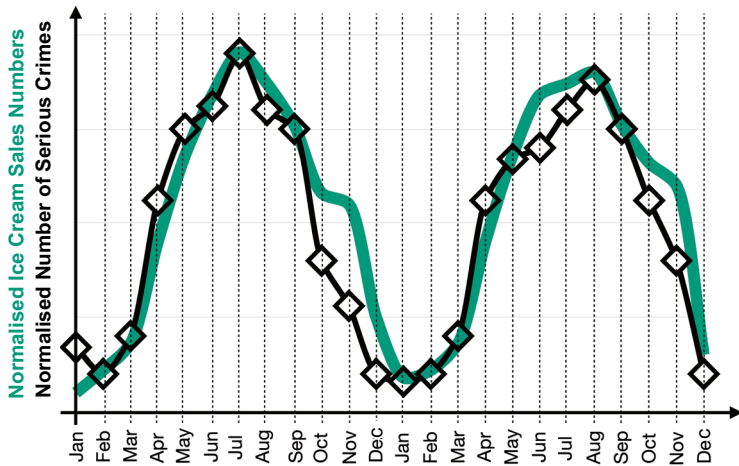


Fig. 1.5 Two different (normalized) time series: sales of ice cream (green) and number of violent crimes (blue) in a city

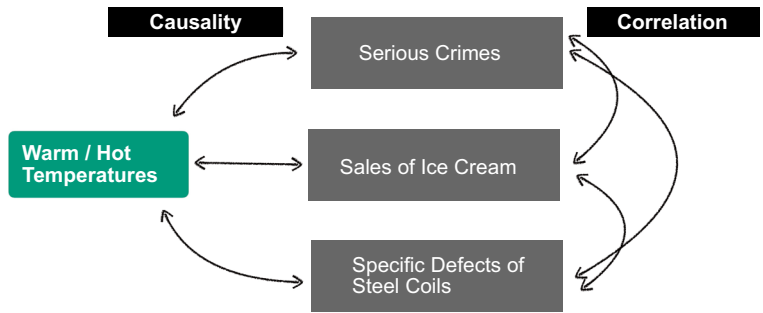


Fig. 1.6 A single cause can generate several similar data series without these being causally related to each other

from the difference between correlation and strong causality. One cannot simply derive a causal dependency from the observation of a correlation in the data. However, there could be a hidden cause for both curves—the causal source—that leads to the visible correlation. In this case, the cause is the temperature.

Fig. 1.6 illustrates the true dependency. High temperatures lead to high crime rates because they lower the tolerance for aggression. Of course, high temperatures also cause ice cream sales to rise because more and more people crave refreshment. In other technical contexts, high temperatures can lead to a lower number of certain defects.

This example is an association task. Starting from the identification of the root cause, we derive a rule for predicting a certain variable. Thus, it is possible to predict the consumption of ice cream or the crime rate by closely observing the temperature.

1.3.4 Industrial Data Mining and the CRISP-DM

The cross-industry standard process for data mining (Cross-Industry Standard Process for Data Mining, CRISP-DM), as presented in [10] among others, is a unified approach to the application of data mining in industrial projects. CRISP-DM is a process model. Such models help to establish a certain methodology in a company systematically.

- **Phase 1. Understanding the business model.** Companies need a special incentive to develop new methods and techniques. They must be profitable. Therefore, it is of great interest to determine the economic benefit and the business model behind the data mining task. Where does the mining activity add value, e.g., through money or energy savings or by avoiding errors or downtime? S. Aggarwal and N. Manuel assess in [2] the requirements from the business model as crucial for successfully implementing data-based solutions in the long term.

It is also important for business understanding to quantify the Return on Investment (ROI). The decisive element here is the time it takes for the initial investment to pay off. Imagine you are spending resources and material costs to advance a data mining project. If this investment only pays off after twenty years or yields a negligible profit, it may not be worth pursuing at all.

The result of Phase 1 must be a set of important **performance indicators** (Key Performance Indicators, KPI) (KPI) that measure the impact of the applied method. The KPIs are used throughout CRISP-DM for monitoring and improving processes.

- **Phase 2. Understanding the data.** Equally important is understanding the data needed to perform a particular data mining task. In the operation of technology companies or production facilities, this is often associated with answering the following questions:
 - What data do we need for the task?
 - What do these data describe? What processes are behind these data?
 - What do the data look like? Here you should manually visualize and check the data.
 - Are the data already collected and if not, what steps are necessary to collect the data?
 - Are additional sensors required?
 - Are the data available in a database? Do the employees need special access for data mining to work with these data?

- What is the quality of the data? Are they complete or do you have to expect missing or incorrect data?
- **Phase 3. Data preprocessing.** Once you understand the data, you can get an idea of how reliable the data are and whether cleaning or selection procedures are necessary or not.
- **Phase 4. Modeling.** This is the practical application of a data mining model, which could mean, for example, applying one of the methods presented in the later chapters on supervised or unsupervised machine learning.
- **Phase 5. Evaluation.** This phase largely corresponds to its abstract analogue in Sect. 1.3.2. Here, the results of the data analysis are critically questioned. An additional aspect in CRISP-DM is the consideration of economic constraints: How well does the data analysis process meet the objectives of a company or production strategic view? The KPIs determined in Phase 1 are used to capture the success and quality of data mining.
- **Phase 6. Deployment.** In this phase, the learned data relationships are practically applied in the company processes. In the process industry, this is the online use during production. This phase brings new difficulties, because now the models gained through data mining must be used to achieve a certain benefit. Either to identify faulty products or process states, or to control the production machines to achieve better quality in advance.
For deployment, it is important to know how long a model is run and how accurate and precise the model is. Especially when automation components depend on the model output, deployment includes all those tasks to make their operation stable and enable an online application in a closed loop.
- **Phase 7. Monitoring.** A critical evaluation of the performance of data mining models is provided for in the CRISP-DM approach as a separate phase. It is a feedback phase for modeling and allows the process to be corrected and improved iteratively over time.
However, not only the model quality is monitored. The performance in relation to the KPIs selected in Phase 1 is also taken into account in order to actually quantify the impact of the data mining application (Fig. 1.7).

The CRISP-DM process models the rather abstract data mining tasks described in a model template, with phases 1 and 2 responsible for the essential steps of data mining preparation, while phase 3 includes activities such as data cleaning and transformation. Each phase contains a series of tasks that must be completed, and the results of each phase serve as input for the next phase. The goal of CRISP-DM is to ensure that all necessary steps are taken to create a high-quality data mining model and that the model is used appropriately.

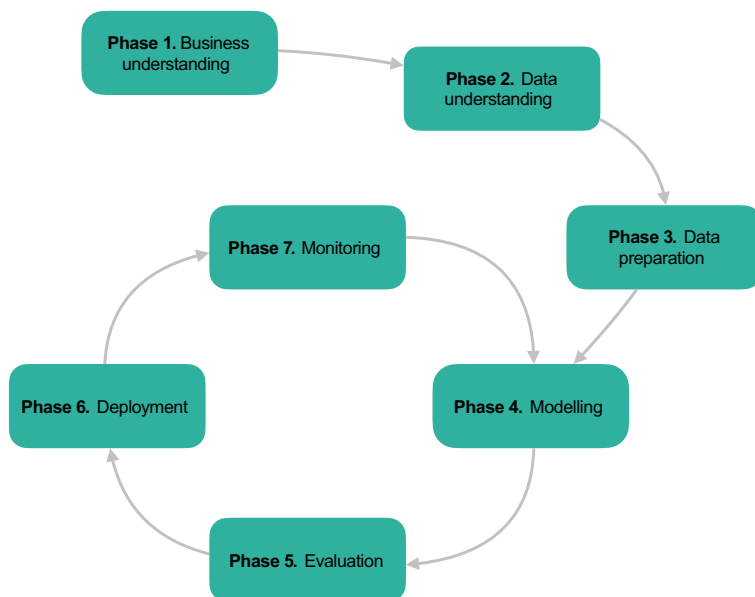


Fig. 1.7 CRISP-DM and its various phases

1.4 Practical Handling of Data in Python

1.4.1 *Programs in this Book*

Jupyter Notebooks

We will be working with Python throughout this book. All programs will be provided to you as so-called Jupyter Notebooks. In Appendix A, we suggest a suitable Python distribution and the associated programming environment. Appendix A also contains a compilation of basic knowledge in Python. It can help you get to grips with the language and offers an introduction for those who are completely unfamiliar with Python. However, we do assume some basic knowledge of programming languages and computer science for this book.

Why Python?

There are many programming languages that can be used to implement machine learning. Lua, C#, C++, Java, Matlab, and Julia are just a few examples. For applications in the technical field, it is particularly important that a language allows non-computer scientists to get started. Python has many advantages here:

For one thing, Python is easy to learn. Supported by a large community on the net, it is considered one of the best languages for programming beginners. It also has a wide syntactic range. Code can be written functionally or object-oriented, depending on the requirements of a project.

Python is an interpreted language. This means we can execute it directly without performing a compilation. With the help of Jupyter notebooks, we can directly check changes to the code for their effect, which promotes testing and simply playing with the code.

If you want to use your own learning methods in industry, it is easy to integrate solutions developed in Python into existing infrastructures. Almost every data management system of larger companies, manufacturing companies or even authorities offers interfaces to this programming language. It runs on both large computer centers and microcomputers like the Raspberry Pi.

1.4.2 Libraries for Python

If you implement algorithms purely in Python, they are often very slow, but can be formulated in a way that is easy to understand. Where speed matters, Python uses special libraries written in C++ code.

Libraries are therefore the component that makes Python powerful. There is a wealth of scientific tools that are tested, proven, and ready to use. These libraries are free and can be used by anyone. The following list is not complete, but reflects the tools we will use more frequently in this book:

- **Numpy.** Numpy [6] is a framework that provides us with various mathematical operations such as vectors, matrices, the Fourier transformation, and a number of tools for data transformation.
- **Matplotlib.** Matplotlib [7] is the gold standard for creating academic diagrams and visualizations in Python. We use this library for plotting.
- **Pandas.** Pandas [11] is a library dedicated to handling datasets, manipulating datasets, and easy saving and loading of data. It offers us an easy way to load, save, and work with data. With Pandas, extremely fast vector and matrix operations are also possible [8]. However, we limit our use of Pandas somewhat and try to get by with the structures of Numpy where possible.
- **Scikit-Learn.** The Scikit-Learn library [9] is a diverse, easy-to-use toolkit for applying almost all methods we introduce in this book.
- **Tensorflow.** Tensorflow [1] is a library from Google for computing with tensors, as needed for building neural networks, for example.
- **Keras.** Keras [3] provides a simple interface for building and working with neural networks that directly builds on Tensorflow. It simplifies the creation of networks and allows you to quickly create solutions.

1.4.3 Explainable Data in Python

The concept of explainability is at the forefront of many works that use algorithms in practice. With this term, we capture the ability to understand a model

or a relationship. How did an algorithm arrive at its result? Why did a computer program make a certain decision? If we can explain the decision-making process in an understandable way, trust in our model increases. This consideration will become important later, especially in the context of learning methods. Are we even able to trust such a method? We will therefore get to know approaches that allow us to look deeper into learning methods and better understand how they work.

Explainability includes a basic understanding of the input variables as well as their significant properties and boundary conditions. In many real database systems, this level of variable description is missing. Sometimes the true meaning of the captured folders is not digitally recorded, at worst only in the head of an employee. This carries enormous risks and greatly complicates practical work.

Units of measurement

A first clear step towards explainability is the consistent consideration of units of measurement. They define the scales on which we measure. Measured variables are a product of their value $\{x\}$ and their unit $[x]$,

$$x = \{x\}[x]. \quad (1.2)$$

Without knowledge of the unit of measurement, the data is practically worthless. When dealing with technical data and process variables, it is therefore crucial to keep a close eye on the units. In some situations, the units of variables can also change over time. Just think of a scale that initially measures in grams and then automatically switches to kilograms for its measuring range when something heavier than 1 kg is placed on it. Consequently, we must always include the units in our data handling. One way to do this is shown in Listing 1.1. Here you see two arrays for t and x as well as a dictionary.

Listing 1.1 Treatment of units

```
1 t = [0,1,2,3] # array for indices
2 x = [1.0,2.0,3.0,4.0] # array for measurement
3 myDictionary = {'Time':t, 'Position':x, '
    Position_Unit':'m', 'Time_Unit':'s'}
```

Here we capture additional information alongside the actual data in the arrays, by including them in the data structure `myDictionary` and storing them as additional fields. Often it is enough to integrate a description in the form of a string into the dictionary.

Units are a basic prerequisite for the **explainability** of a variable.

Unfortunately, units are completely missing or are only visible through secondary tables in surprisingly many real process databases. An automatic check of the units is then difficult.

During data collection, changes in units should definitely be avoided. They increase the complexity of preprocessing without generating added value. Why are unit jumps observed in datasets at all? The main reason lies in the display of numbers. Digital measuring devices always try to display their values on an optimal scale. However, the data indication optimized for a display is not always optimal for data storage. If you agree on consistency in the unit, you save time and effort in your later evaluation. You also eliminate a known source of error.

Taking uncertainties into account

In the case of measured quantities, the inclusion of uncertainties helps us to further increase our understanding of a variable. It provides us with a quantitative way to describe our confidence in the respective quantity. The absolute uncertainty u_x for any quantity x , would describe the measured value x_m in the following way,

$$x_m = x \pm u_x. \quad (1.3)$$

The absolute uncertainty is related to a relative uncertainty by

$$\varepsilon_x = \frac{u_x}{x_m}. \quad (1.4)$$

These two uncertainties can be used in our dictionary to describe the data, as shown in the following extension of our code example:

Listing 1.2 Example for adding stochastic variables

```
1 t = [0,1,2,3] # array for indices
2 x = [1,2,3,4] # array for measurement
3 myDictionary = {'Time':t, 'Position':x, '
    Position_Unit':'m', 'Time_Unit':'s', '
    Position_Absolute_Uncertainty':0.05, '
    Time_Absolute_Uncertainty':0.01}
```

Knowing uncertainties, communicating them to the user, and ultimately always considering them in the larger context, is a first step in building trust in an algorithm.

The informed indication of uncertainties increases the **trust** in data. Agree on a form to indicate these uncertainties, absolute or relative, and consistently maintain this form for your data collection.

Summary

In this first chapter, we have dealt with data. Our goal is to create models from data using machine learning. With these models, we can analyze and improve technical processes. To do this, we first had to show ways in which we can describe, characterize, and structure data. We have seen that in addition to fully structured and unstructured data, there is also a semi-structured type.

Furthermore, first, simple steps in Python were introduced. Here we have seen that the systematic consideration of units and the treatment of uncertainties play a major role in data evaluations. Without knowledge of their unit, a physical quantity is incomprehensible. Without knowledge of the uncertainty, we do not know how far we can trust the data value.

Tasks

1.1 Characterize the following data collections!

- Phone book
- Calendar entries
- Address book for contacts
- Electricity consumption

1.2 Why is focusing important in data mining? Can you provide reasons why not all possible data is always considered?

1.3 What type of scale is present in the following data sets?

- Scale for measuring length
- Rowing motion in an airplane
- Pressure measurement in hPa
- Collection of fish names
- Wood colors
- Camera images

1.4 Let's assume you have measured rolling forces in an aluminum plant. What additional information do you need in order to draw conclusions about the quality from the data?

1.5 You work in a sawmill and your saws wear out over time. a) What variables would you record to trace the cause? b) How would you measure the variables? c) What types of data and scales are involved with your data?

References

1. M. Abadi, A. Agarwal, P. Barham, E. Brevdo, Z. Chen, C. Citro, G. S. Corrado, A. Davis, J. Dean, M. Devin, S. Ghemawat, I. Goodfellow, A. Harp, G. Irving, M. Isard, Y. Jia, R. Jozefowicz, L. Kaiser, M. Kudlur, J. Levenberg, D. Mané, R. Monga, S. Moore, D. Murray, C. Olah, M. Schuster, J. Shlens, B. Steiner, I. Sutskever, K. Talwar, P. Tucker, V. Vanhoucke, V. Vasudevan, F. Viégas, O. Vinyals, P. Warden, M. Wattenberg, M. Wicke, Y. Yu, and X. Zheng. TensorFlow: Large-scale machine learning on heterogeneous systems, 2015. <https://www.tensorflow.org/>. Software available from tensorflow.org.
2. S. Aggarwal and N. Manuel. Big data analytics should be driven by business needs, not technology. *McKinsey & Co.*, June 2016.

3. F. Chollet et al. Keras, 2015. <https://github.com/fchollet/keras>.
4. L. Fahrmeir, R. Künstler, I. Pigeot, and G. Tutz. *Statistik*. Springer, 1997.
5. J. Hamilton. *Time Series Analysis*. 1992.
6. C. R. Harris, K. J. Millman, S. J. van der Walt, R. Gommers, P. Virtanen, D. Cournapeau, E. Wieser, J. Taylor, S. Berg, N. J. Smith, R. Kern, M. Picus, S. Hoyer, M. H. van Kerkwijk, M. Brett, A. Haldane, J. F. del Río, M. Wiebe, P. Peterson, P. Gérard-Marchant, K. Sheppard, T. Reddy, W. Weckesser, H. Abbasi, C. Gohlke, and T. E. Oliphant. Array programming with NumPy. *Nature*, 585(7825): 357–362, Sept. 2020. <https://doi.org/10.1038/s41586-020-2649-2>.
7. J. D. Hunter. Matplotlib: A 2d graphics environment. *Computing in Science & Engineering*, 9(3): 90–95, 2007. <https://doi.org/10.1109/MCSE.2007.55>.
8. T. pandas development team. pandas-dev/pandas: Pandas, Feb. 2020. <https://doi.org/10.5281/zenodo.3509134>.
9. F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg, J. Vanderplas, A. Passos, D. Cournapeau, M. Brucher, M. Perrot, and E. Duchesnay. Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12: 2825–2830, 2011.
10. C. Shearer. The crisp-dm model: The new blueprint for data mining. *Journal of Data Warehousing*, 5(4), 2000.
11. Wes McKinney. Data Structures for Statistical Computing in Python. In Stéfan van der Walt and Jarrod Millman, editors, *Proceedings of the 9th Python in Science Conference*, pages 56–61, 2010. <https://doi.org/10.25080/Majora-92bf1922-00a>.